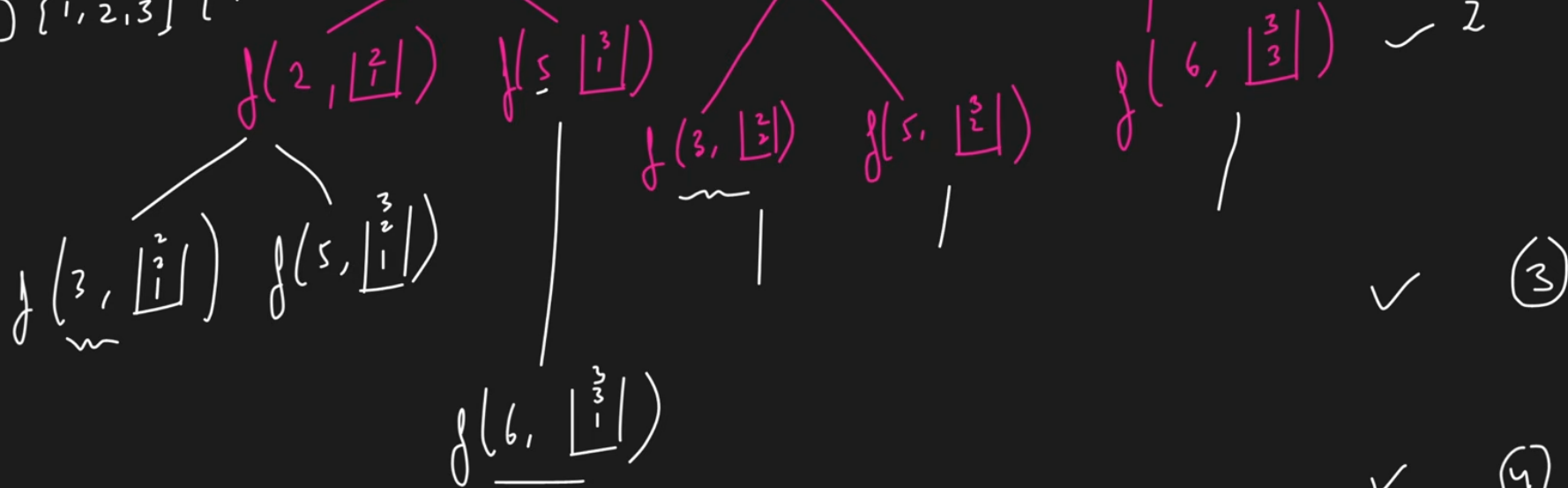


L11. Subset Sum II | Leetcode | Recursion

2.30 [1, 2, 2, 2, 3, 3]
0 1 2 3 4 5

{}, {1}, {2}, {3}
{1,2} {1,3} {2,2} {2,3}

{1,2,2} {1,2,3} {1,3,3}



✓ (3)

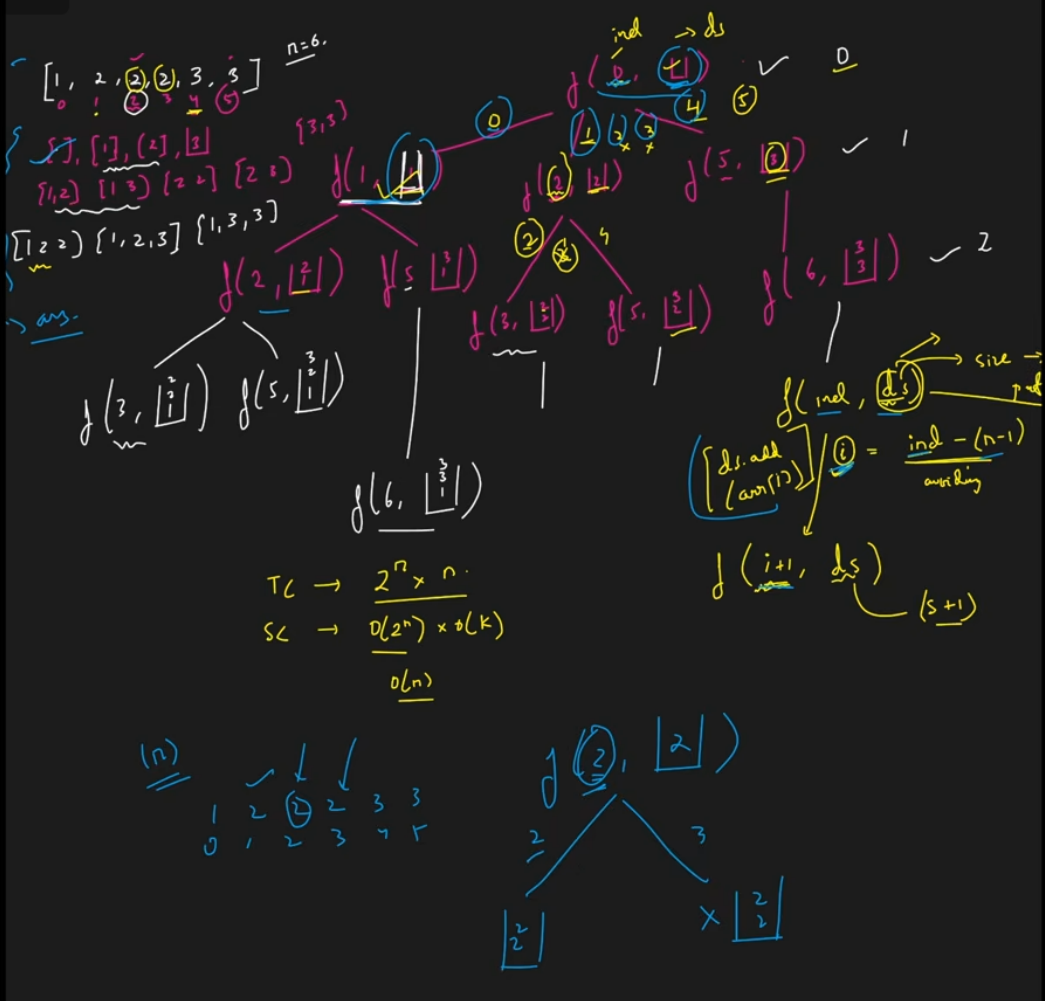
✓ (4)

✓ (5)

✓ (6)

L11. Subset Sum II | Leetcode | Recursion

2.30



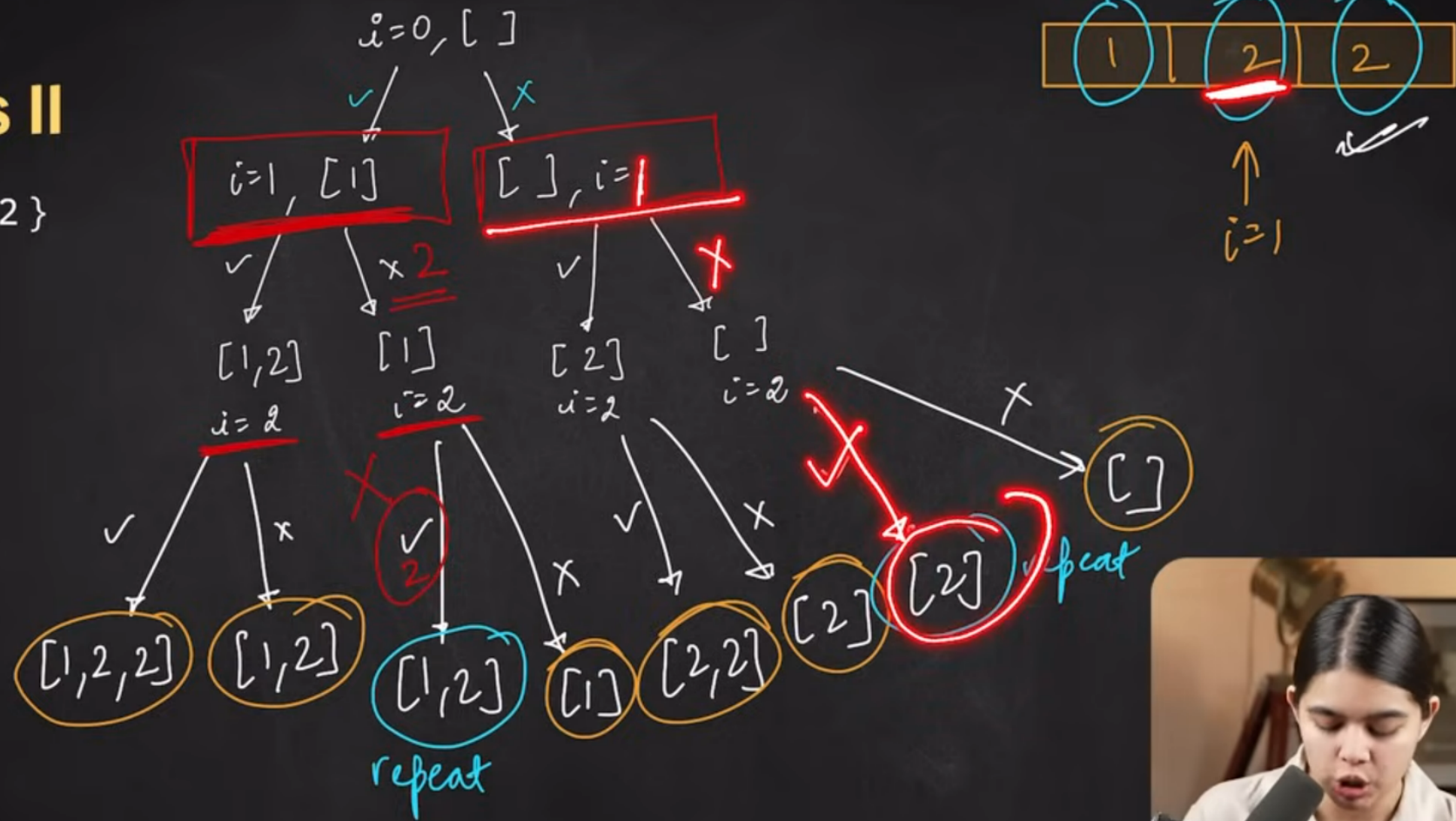
```

1 class Solution {
2 private:
3     void findSubsets(int ind, vector<int> &nums, vector<int> &ds, vector<vector<int>> &ans) {
4         ans.push_back(ds);
5         for(int i = ind; i < nums.size(); i++) {
6             if(i != ind && nums[i] == nums[i-1]) continue;
7             ds.push_back(nums[i]);
8             findSubsets(i+1, nums, ds, ans);
9             ds.pop_back();
10        }
11    }
12 public:
13    vector<vector<int>> subsetsWithDup(vector<int> &nums) {
14        vector<vector<int>> ans;
15        vector<int> ds;
16        sort(nums.begin(), nums.end());
17        findSubsets(0, nums, ds, ans);
18        return ans;
19    }
20 };

```

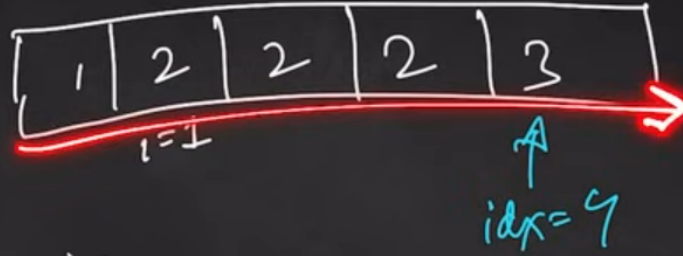
Subsets II

arr = {1, 2, 2}



Subsets II

arr = {1, 2, 2}



```
void pS(arr, ans, i, allSubsets) {  
    [ if (i == n) allSubsets(ans) return; ] RC
```

```
    ans.pushback(arr[i]);  
    pS(arr, ans, i+1, allSubsets);  
    ans.pop-back();  
}
```

skipping the duplicate values

```
    idx = i+1  
    while (idx <= n && A[idx] == A[idx-1])  
        idx++
```

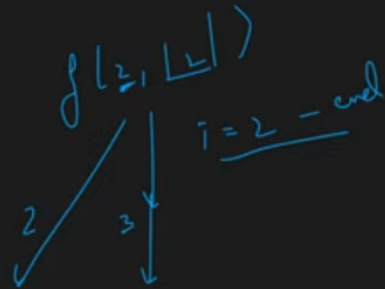
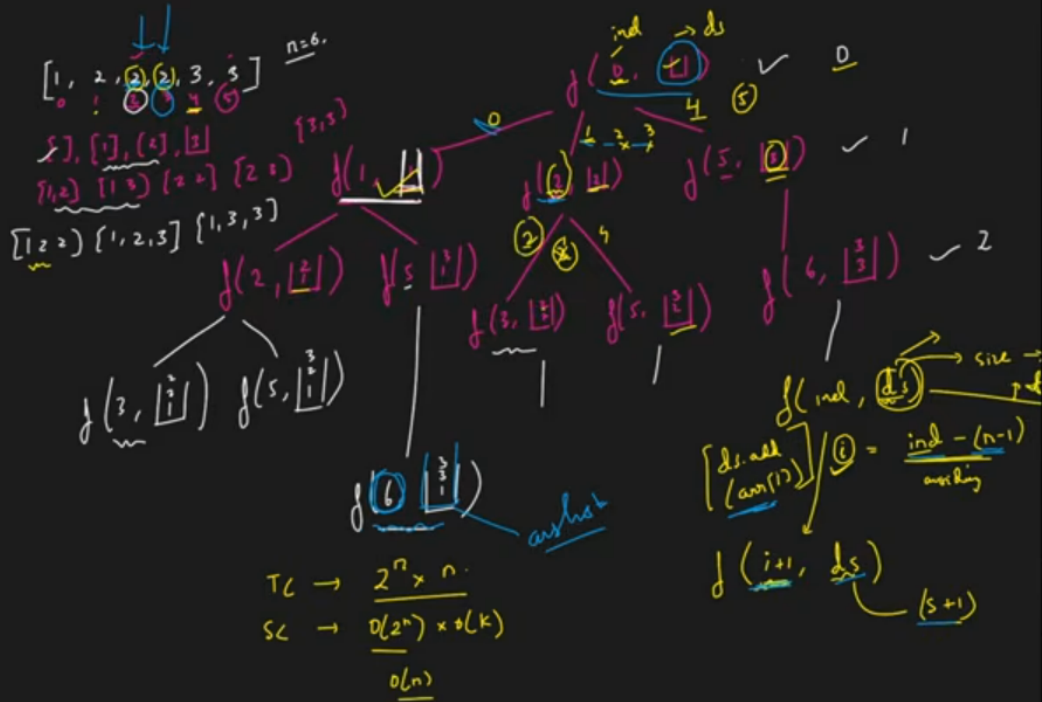
```
    pS(arr, ans, idx, allSubsets)
```

}

① sort the arr

② calc subsets





```

1 class Solution {
2
3     public void findSubsets(int ind, int[] nums, List<Integer> ds, List<List<Integer>> ansList) {
4         ansList.add(new ArrayList<>(ds));
5         for(int i = ind; i < nums.length; i++) {
6             if(i != ind && nums[i] == nums[i-1]) continue;
7             ds.add(nums[i]);
8             findSubsets(i+1, nums, ds, ansList);
9             ds.remove(ds.size() - 1);
10        }
11    }
12
13    public List<List<Integer>> subsetsWithDup(int[] nums) {
14        Arrays.sort(nums);
15        List<List<Integer>> ansList = new ArrayList<>();
16        findSubsets(0, nums, new ArrayList<>(), ansList);
17        return ansList;
18    }
19 }
  
```

</> C++

means:

"This is the FIRST choice I'm trying at this level."

And:

C++

```
i > index
```

means:

"I've already tried an earlier choice at this level."

Bingo.

So now we say:

If the current value is the same as the previous value **AND** I'm not the first choice of this level, skip it.

That becomes:

C++

```
if(i > index && nums[i] == nums[i-1])
    continue;
```

4. Now the condition almost invents itself

We wanted:

Requirement A

Current value must equal previous value:

+ | Ask anything

 **Think**



arr = [1, 2, 2]

f(int idx/start)

